KJS India Platform: Master Build Prompt

Oct 23, 2026 · @vineet pandey

1. Project and current state

KJS India platform: a public website for B2B importers (SEO-first, captures every lead), an employee portal and an admin console. Go-live **Fri 11 Dec 2026** (latest 18 Dec). Buyer intelligence and outreach are Stage 2 (2027).

Repo: github.com/vineet110/kjs_m, local `C:\VINEET_WORK\KJS_v1`, folders `frontend/` and `backend/`.

Stack in use

Part	Choice
Frontend	Next.js 14.2 App Router, next-intl 3.26 (English only at launch), Tailwind
Backend	FastAPI, Python 3.11, SQLAlchemy, Alembic, pydantic-settings
Database	Neon Postgres (dev/staging); company server later via `pg_dump` / `pg_restore`
Files	Cloudflare R2 (S3-compatible); local folder in dev, same module
AI (Koko)	Groq `openai/gpt-oss-120b` behind an `LLM_PROVIDER` switch; Anthropic can plug in later
Auth	Session cookie (httpOnly, hash stored), CSRF header, scrypt passwords
Roles	admin + employee, with per-employee permission keys
Charts	Native charts from real DB data; Superset after go-live
Brand	Existing kjsindia.com look; 21 assets already in `frontend/public/images/kjs/`

Already working: public site (home, categories, products, capabilities, contact, enquiry form, SEO, sitemap, JSON-LD), Koko chat with lead capture, staff sign-up and approval, login, permissions, employee leads list, admin dashboard, products, audit log, holidays API, working-days calendar, health endpoints, 52 tests, CI workflow.

Decisions: D1 Next.js · D2 English first · D3 gate actions only · D4 Groq for now · D5 admin + employee only · D6 native charts · D15 Neon now · D16 Cloudflare R2 · D17 keep `frontend/` and `backend/`.

2. Client view: public website

Set **Decision** on each row (Must, Should, Later, Not needed) and add rows for anything new. Routes sit under `/[locale]`, English only for now.

Page / feature	Route	What it does	Status	Decision
Home	`/`	Hero, best sellers, categories, capabilities, certifications, contact strip	Done	Must
Category pages	`/categories/[slug]`	Products in a category with short intro	Done	Must
Product pages	`/products/[slug]`	Specs, images, enquiry button	Done	Must
Capability pages	`/capabilities/[slug]`	Stick-pack factory, processing, packaging, warehousing, quality	Done	Must
Contact + enquiry form	`/contact`	Form with consent and honeypot, rate limited, saves a lead	Done	Must
Koko chat	all pages	Answers from facts and catalogue, captures buyer leads	Done	Must
SEO basics	–	Metadata, sitemap, robots, JSON-LD	Done	Must
Offers ticker	home	Scrolling offers set by admin	Not started	Should
Gated price sheet / spec PDF / sample / quote	product pages	Short form once, then remembered; creates a lead	Not started	Must

Page / feature	Route	What it does	Status	Decision
Private-label configurator	`/private-label`	Pick product, pack, quantity, market; shows indicative price	Not started	Should
Partner programme page	`/partners`	Agents and forwarders apply to refer buyers	Not started	Should
WhatsApp button	all pages	Click to chat	Not started	Should
More languages (AR, ES, FR)	`/ar` etc.	Translated site, Arabic right-to-left	Not started	Later
Importer landing pages	`/importers/[country]/[product]`	One page per market and product (Stage 2)	Not started	Later
Client login	`/account`	Buyers track quotes, samples, orders (Stage 2)	Not started	Later
Your feature			▼	▼

3. Employee portal

Routes under `/employee`, login required.

Page / feature	Route	What it does	Status	Decision
Login and sign-up request	`/employee/login`	Sign in; new staff wait for admin approval	Done	Must
Dashboard with my leads	`/employee/dashboard`	My leads, change status (new, contacted, closed)	Done	Must

Page / feature	Route	What it does	Status	Decision
SLA timers on leads	dashboard	Respond within 20 min, resolve within 1 h; overdue shown and escalated to admin	Not started	Must
Availability toggle	dashboard	"Receiving tasks" on or off	Not started	Must
Attendance	`/employee/attendance`	Check in / out, week view, hours, regularization request, holidays	Not started	Must
My tasks	`/employee/tasks`	Tasks from all sources, due dates, notes, add own task	Not started	Must
Auto-assigned batches	`/employee/batches`	Records given to me automatically; work through and mark done	Not started	Must
Order boards (my steps)	`/employee/order-boards`	Boards I am on: my step, its tasks, files, progress vs plan	Not started	Must
Notifications	header	In-app, email, WhatsApp (opt-in)	Not started	Must
Profile	`/employee/me`	My details, password, notification settings	Not started	Should
Buyer account page	`/employee/buyers/[id]`	Buyer score, contacts, timeline (Stage 2)	Not started	Later

Page / feature	Route	What it does	Status	Decision
Your feature			▾	▾

4. Admin console

Routes under `/admin`, admin role or the matching permission key.

Page / feature	Route	What it does	Status	Decision
Dashboard	`/admin/dashboard`	Real numbers: leads, staff, products, recent activity	Done	Must
Staff approval and permissions	`/admin/team`	Approve sign-ups, grant permission keys	Done	Must
Products	`/admin/products`	Add and edit categories, products, specs, images	Done	Must
Audit log	`/admin/audit`	Who changed what and when; filter	Done	Must
Holidays	`/admin/holidays`	Holiday list for attendance and order boards (API done, page needed)	Partly done	Must
All leads and SLA settings	`/admin/leads`	Every lead, owner, SLA status; reassign	Not started	Must
Team attendance	`/admin/team-attendance`	Attendance reports, regularization approvals	Not started	Must
Approvals inbox	`/admin/approvals`	Sign-ups, regularizations, file delete requests in one list	Not started	Must

Page / feature	Route	What it does	Status	Decision
Auto-assign settings	`/admin/auto-assign`	Categories per employee, batch size, max per day, preview, run now	Not started	Must
Order boards	`/admin/order-boards`	Board code, deadline, steps by % or days on working days, members, task templates, files, alerts at 50/85/100%	Not started	Must
File uploads	used by boards, products	Upload to Cloudflare R2 (local folder in dev)	Not started	Must
Alerts	`/admin/alerts`	Email and WhatsApp alert rules and log	Not started	Must
Analytics	`/admin/analytics`	Native charts from real data	Not started	Should
Website content and offers	`/admin/website`	Edit page text, SEO titles, offers ticker	Not started	Should
Koko settings	`/admin/koko`	Facts file, tone, chat transcripts	Not started	Should
Settings	`/admin/settings`	View effective config and edit allowed config.ini values	Not started	Should
Buyer intelligence	`/admin/bi/*`	Sources, markets, discovery, de-dupe, scoring (Stage 2)	Not started	Later
Outreach	`/admin/outreach/*`	Campaigns, approval of emails, mailboxes (Stage 2)	Not started	Later
Your feature			▼	▼

5. Twelve-Factor App

Every service (frontend, backend, worker, scheduler) follows these.

#	Factor	In this project
1	Codebase	One repo, same code deployed to dev, staging, production
2	Dependencies	Locked in `requirements` / `package-lock.json`; Docker images hold everything
3	Config	Secrets and per-environment values in `.env`; tunable settings in `config/*.ini` (section 6)
4	Backing services	Postgres, R2, Groq, SMTP attached by URL or key; swap without code changes
5	Build, release, run	CI builds; release = image + config; never edit code on the server
6	Processes	Stateless; sessions, jobs and files live in Postgres or R2, not in memory or local disk
7	Port binding	Frontend on 3000, backend on 8000; nginx only routes
8	Concurrency	Scale by running more processes (more workers), not bigger ones
9	Disposability	Fast start, clean shutdown on SIGTERM, jobs safe to re-run
10	Dev/prod parity	Same Postgres version and Docker images everywhere
11	Logs	Written to stdout as JSON (section 7)
12	Admin processes	Migrations, seed, one-off fixes as commands (`make migrate` , `python -m app.seed`)

6. Config: .env and config.ini

Secrets stay in `.env`; settings we may want to tune go in `config/*.ini`, and editing an ini file applies within a few seconds without a restart.

What	Where	Examples
Secrets, URLs, per-environment values	`.env` (pydantic-settings, as now)	`DATABASE_URL`, `GROQ_API_KEY`, `R2_*`, `SESSION_SECRET`, `LLM_PROVIDER`
Tunable settings	`config/*.ini`, live reload	SLA minutes, batch size, rate limits, log levels, feature on/off

Files:

```
config/
  app.ini        # site name, timezone, page sizes, rate limits
  features.ini   # on/off: koko, offers_ticker, configurator, partners
  sla.ini        # respond_minutes, resolve_minutes, escalate_after_minutes
  assign.ini     # batch_size, max_batches_per_day, working_hours
  alerts.ini     # thresholds 50/85/100, channels
  koko.ini       # model, temperature, max_tokens
  logging.ini    # level per module, JSON or pretty
```

Example:

```
; config/sla.ini
[sla]
respond_minutes = 20
resolve_minutes = 60
escalate_after_minutes = 10
```

How it works:

1. One loader in `backend/app/core/config/` reads the ini files with Python's built-in `configparser` and checks them with pydantic models.
2. It checks file modified-times every few seconds (no extra package). On change: validate, then swap. If invalid, keep the old values and log an error naming the file and key.

3. Code reads values when it needs them (`settings().sla.respond_minutes`), not once at import, so changes apply live.
4. Order: env var override (`KJS__SLA__RESPOND_MINUTES`) → ini file → default in code.
5. Each reload logs `config.reloaded` with the keys that changed.
6. Frontend gets only safe values through `GET /api/public/config`; it never reads ini files.
7. In Docker, mount `config/` as a volume so editing on the server works without a rebuild.

7. Logger

Every request and job can be traced from one `request_id` in the logs.

- **Backend:** Python's built-in `logging` with a small JSON formatter, written to stdout. Pretty text in dev, JSON in staging and production (set in `logging.ini`).
- **Frontend:** a small server-side helper in `frontend/src/lib/logger.ts` that prints JSON lines; no extra package.
- **Request ID:** Next.js sends `X-Request-ID` to the backend; a middleware puts it on every log line, returns it in the response, and adds it to the error shape `{code, message, details, request_id}`. Jobs log their `job_run_id`.
- **Fields on each line:** `ts` (UTC), `level`, `service`, `event` (for example `lead.created`), `request_id`, `user_id`, plus `route`, `status`, `duration_ms` for requests.
- **Levels live:** change a module's level in `logging.ini` (for example `app.services.lead = DEBUG`) and it applies on the next reload.
- **Hide secrets:** never log passwords, tokens, cookies or API keys; mask emails and phones (`om***@alnoor.ae`).
- **What to log:** login success and failure, permission denied, lead created, SLA breach, job start/finish/fail, config reload, errors with stack trace (once, where handled).
- `make logs` tails everything; `make trace ID=<request_id>` shows one request across frontend and backend.
- The audit log in the database stays separate; it records business changes, logs are for debugging.

8. Open items

- [ ] D9: what "records" in an auto-assign batch are
- [ ] D18: does the company expect Stage 2 features from the demo video by 11 Dec?
- [ ] D19: public USD prices, or price sheet via form?
- [ ] Production hosting, domain and DNS access; IT contact
- [ ] I9: stick-pack fill range (0.9–3 g vs 18 g)
- [ ] Slow API (~1.4 s): move Neon to Singapore or use local Docker Postgres for dev
- [ ] Rotate Groq key and Neon password (pasted in chat) before go-live
- [ ] From KJS: product list, price bands, employee list and holidays, Koko FAQs, order-board templates, official brand files
- [ ] WhatsApp Business verification

9. NOTE:
timeline is just 2 weeks to finish the whole phase 1 .
then followed by the production and all , where we see the actual server , deployment etc and other migrations. so just have a single list where we list each feature one by one , and keep checking them as we finish them . this way it will be easy .ok? so make in docs, checklist.md and list all one below other with heading of that or subheading followed by the features one by one .